Multilayer Perceptrons and the Backpropagation algorithm

Contents

3.1 Limitations of Perceptrons **1**
3.1.1 Solving the XOR problem 2

3.2 Feedforward MLP **4**
3.2.1 Navigating through the indices 4

3.3 MLPs are universal function approximators **7**

3.4 Training a neural network **7**
3.4.1 Recap of gradient-based learning 7
3.4.2 Computing gradients for an MLP 8

3.5 The Backpropagation algorithm **9**
3.5.1 Forward and backward propagation 10
3.5.2 Example: MLP with fully connected layers 12
3.5.3 Summary of the Backpropagation for gradient descent 15

3.6 Symmetries **16**
3.6.1 Permutation of indices within consecutive layers 16
3.6.2 Sign-reversal across layers 16

3.1 Limitations of Perceptrons

Connectionist neurons, or perceptrons, are limited in the variety of functions they are able to fit. When dealing with classification problems, a perceptron can only find a linear separation between any two classes. Irrespective of the neuron's non-linear activation function, the perceptron is regarded as a *linear classifier*. Whether something falls on one side of the decision boundary or the other, is entirely based on applying a linear filter (i.e. $\underline{\mathbf{w}}^\top \underline{\mathbf{x}}$). The non-linearity $f(h)$ merely controls the value range of the neuron's response $(\{0,1\},(-1,+1),\dots)$ which helps in interpreting the neuron's response.

If observations for two different classes are distributed such that one cannot draw a line to separate them, then the two classes are not linearly separable. In this case, the perceptron will fail to find a suitable classification boundary between the two classes.

Consider the following binary classification problems with $\underline{\mathbf{x}} \in \mathbb{R}^2$.

Q 3.1.1: *Can you find a line that separates the two classes for each case?*

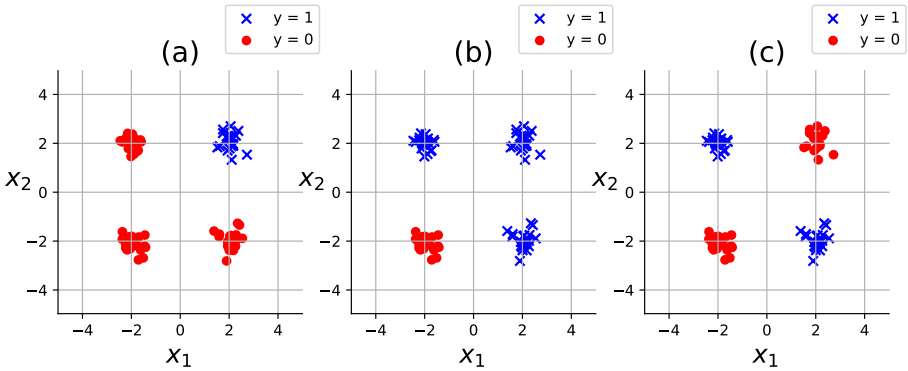


Figure 3.1.1: (a) points are classified according to the AND function, (b) points are classified according to the OR function, (c) points are classified according to the XOR function.

In Figure 3.1.1, particularly (a) and (b), it is possible to draw a line that separates the classes. Therefore, the AND and OR functions are linearly separable. A perceptron is capable of finding such a separating line. However, this does not apply to the third case, for the XOR function. It is impossible to find a single line that will separate the classes. The XOR function is not linearly separable.

Q 3.1.2: *Can we solve the XOR problem with multiple perceptrons? How?*

- Yes, think of it as a divide and conquer approach. We split the XOR problem into multiple sub-problems. A perceptron is used to solve each sub-problem.

If you're familiar with Boolean algebra, you might recognize the following expression for the XOR function:

$$\mathrm{XOR}(x_1, x_2) = (\, x_1 \text{ AND } \overline{x}_2 \,) \;\; \text{OR} \;\; (\, \overline{x}_1 \text{ AND } x_2 \,) \tag{3.1.1}$$

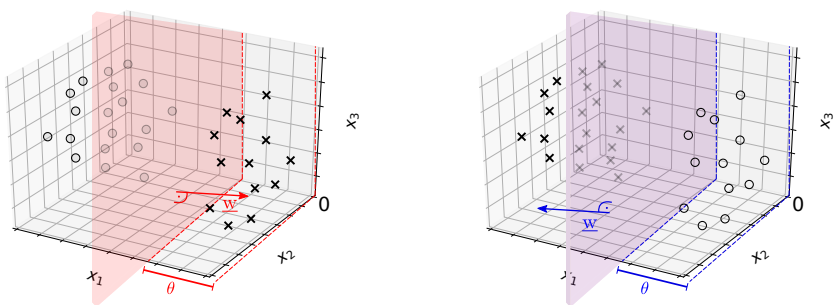


Figure 3.1.2: Inverting a neuron's response by flipping the hyperplane

For instance, the first perceptron s_1^1 is tasked to separate the bottom-right cloud of points from the rest. A second perceptron s_2^1 is used to separate the top-left cloud from the rest. A third perceptron s_1^2 will then use the responses of both and respond to "is **only one** of the two perceptrons **on**?".

Fig. 3.1.4 illustrates this approach. One need only recognize that each sub-problem is linearly separable.

3.1.1 Solving the XOR problem

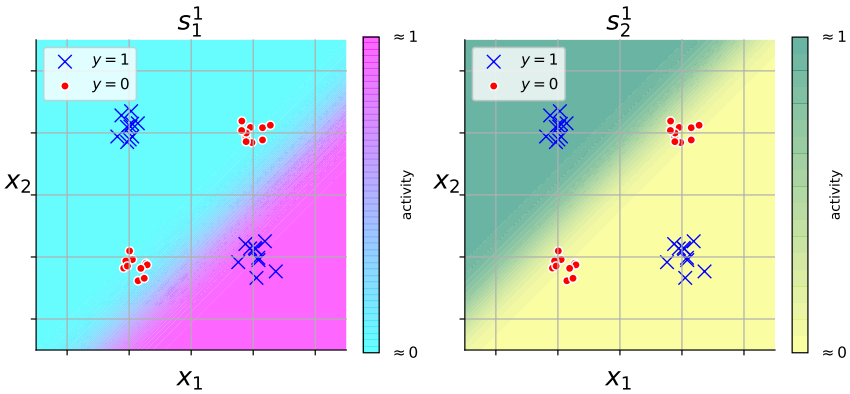


Figure 3.1.3: Solving sub-problems of the XOR problem.

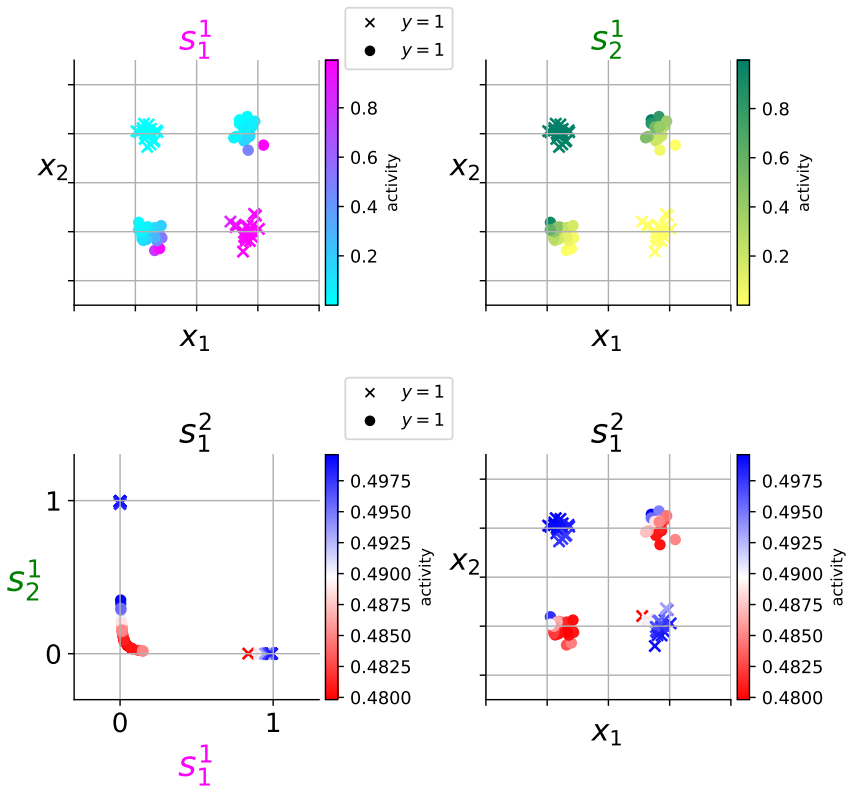
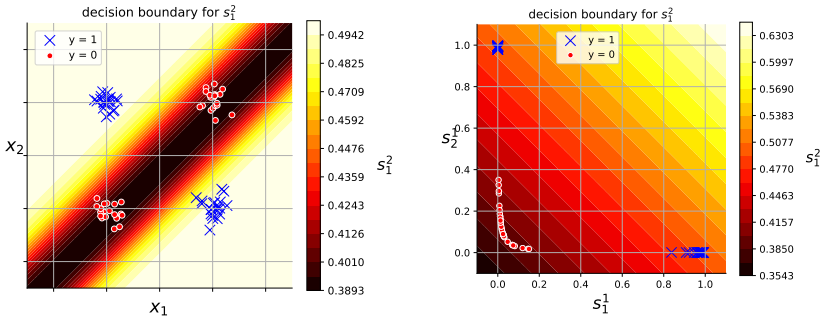


Figure 3.1.4: Neuron responses for the XOR problem.

Fig. 3.1.5a visualizes the response of the final neuron s_1^2 for different values of x_1 and x_2. The space is partitioned into one region in which the activity is < 0.5 and two disjoint regions (top-left & bottom-right) in which the activity is > 0.5.



(a) (b)

Figure 3.1.5: Identifying the decision boundaries.

What we are essentially describing is a Multilayered perceptron (MLP) with an architecture as illustrated in Fig. 3.1.6. This MLP is made up of an output layer with a single output neuron s_1^2, and one hidden layer with two hidden neurons, s_1^1 and s_2^1 (the superscript denotes the layer index, the subscript denotes the neuron index within its layer). The terms "neurons" and "nodes" are treated as synonyms.

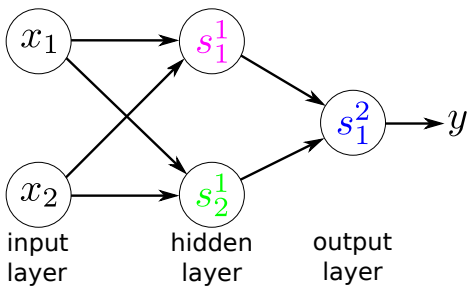


Figure 3.1.6: simplified MLP architecture

3.2 Feedforward MLP

3.2.1 Navigating through the indices

The nodes in a feedforward neural network form a directed acyclic graph (DAG). There are no connections that feed back to a neuron in an earlier layer. A neuron can only be connected to another neuron that lies in a deeper layer in the network. We will eventually cover neural architectures that allow for feedback connections such as recurrent neural networks. Fig. 3.2.1 is an example MLP architecture (simplified by omitting bias nodes).

Each node in the network is a connectionist neuron. This implies that every individual neuron extracts some feature from the "input" it receives. The "input" to a neuron can come from the observed data $\underline{\mathbf{x}}$ or it could be the response of another neuron from an earlier layer in the network. This also implies that *for every neuron* their exists a hyperplane which is represented by some weights $\underline{\mathbf{w}}$ and a bias θ.

s_j^v denotes the *activity* of the neuron j-th in layer v, while $s_i^{v'}$ denotes the *activity* of the neuron i-th in layer v'. h_j^v and $h_i^{v'}$ denotes the *total input* for neuron (v, j) and neuron (v', i), respectively. The activity $s_i^{v'}$ is obtained by applying a transfer function $f_i^{v'}(\cdot)$ to the total input $h_i^{v'}$ of neuron (v', i):

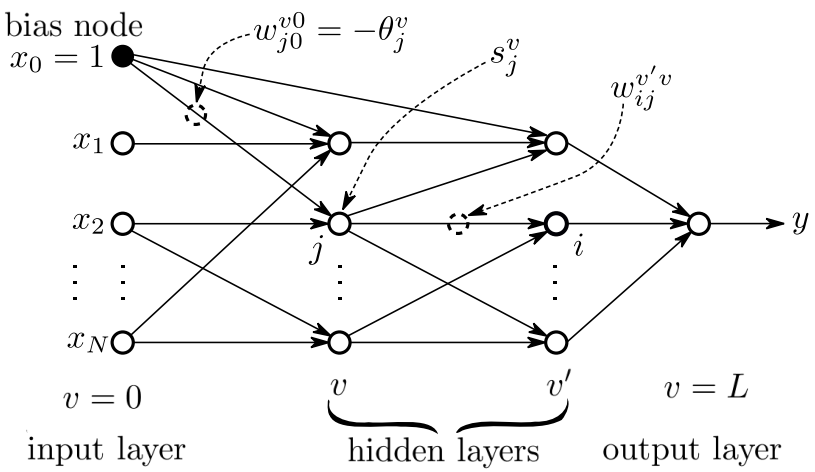


Figure 3.2.1: MLP architecture

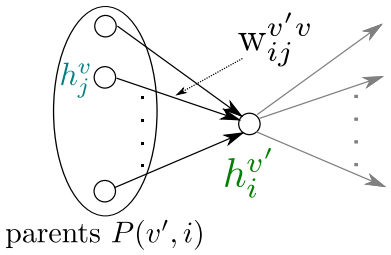


Figure 3.2.2: Computing the total input for neuron (v', i)

$$s_i^{v'} = f_i^{v'}(h_i^{v'}) \tag{3.2.1}$$

where common choices for $f_j^v(\cdot)$, depending on the layer, can be:

- the identity function: $f_j^v(h) := h$. Often for the input layer, i.e. $s_j^0 = f_j^0(h_j^0) = f_j^0(x_j) = x_j$
- logistic sigmoidal or $\tanh(h)$. Common for hidden neurons and output neurons (for classification tasks).

v is a literal that is used to denote a specific layer in the network. $v = 0$ describes the *input layer* which holds the observations $\underline{\mathbf{x}}$. $v = L$ describes the *output layer* of the MLP. All the layers in between, if any, are referred to as *hidden layers*. For example, the scalar output of an MLP can be referred to using:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L) \tag{3.2.2}$$

v', v'' are used to describe layers relative to the current layer v. $v' = v + 1$ and $v'' = v' + 1 = v + 2$ is a very common but this depends on how exactly any two nodes are connected. Therefore, talking about layers v, v' and v'' needs to be put into the context of the connections between neurons. To elaborate:

$w_{ij}^{v'v}$ measures the strength of the connection <u>from</u> (v, j) <u>to</u> (v', i). Seeing $v'v$ in the superscript of the weight tells us that the two neurons are directly connected (1 hop).

w_{j0}^{v0} measures the bias $-\theta_{j0}^{v0}$ for neuron (v, j).

To formulate how neuron (v', i) processes its input requires identifying the set of parent nodes $P(v', i)$ of neuron (v', i). The weighted sum of the parent activations yields the total input $h_i^{v'}$:

$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big), \tag{3.2.3}$$

We now assume an architecture with *fully connected layers*. This implies that every node in layer v is connected to all nodes in the subsequent layer v' (except for the bias node in v' because it has no parents). This type of connectivity allows us to simplify the notation as it implies that the set of parents $P(v',\, i)$ is essentially a vector of all activations in layer v:

$$P(v',i) := (\underbrace{s_0^v}_{\substack{=1\\ \text{(bias)}}}, s_1^v, \ldots, s_j^v, \ldots, s_{N_v}^v)^\top, \tag{3.2.4}$$

where N_v is the number of hidden nodes in layer v.

From this follows:

$$\begin{aligned}
h_i^{v'} &:= \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big) && (3.2.5)\\
h_i^{v'} &\overset{\substack{\text{fully}\\ \text{conn.}}}{=} \sum_{j=0}^{N_v} w_{ij}^{v'v}\, f_j^{v}\big(h_j^{v}\big) && (3.2.6)\\
&= \underline{\mathbf{w}}_i^{v'v\,\top} \cdot \underline{\mathbf{f}}^v\big(\underline{\mathbf{h}}^v\big) && (3.2.7)\\
&= \underline{\mathbf{w}}_i^{v'v\,\top} \cdot \underline{\mathbf{s}}^v && (3.2.8)\\
& && (3.2.9)
\end{aligned}$$

here $\underline{\mathbf{w}}_i^{v'v} := \big(\overbrace{w_{i0}^{v'v}}^{\text{bias}}, w_{i1}^{v'v}, \ldots, w_{iN_v}^{v'v}\big)^\top \in \mathbb{R}^{N_v+1}$

Consequently, to compute the total input for all $N_{v'}$ neurons in the layer v we introduce the weight matrix $\underline{\mathbf{W}}^{v'v} := \big(\underline{\mathbf{w}}_1^{v'v\,\top}, \underline{\mathbf{w}}_2^{v'v\,\top}, \ldots, \underline{\mathbf{w}}_{N_{v'}}^{v'v\,\top}\big)$:

$$\underline{\mathbf{W}}^{v'v} = \begin{pmatrix} | & | & & | & & | \\ \underline{\mathbf{w}}_1^{v'v} & \underline{\mathbf{w}}_2^{v'v} & \cdots & \underline{\mathbf{w}}_i^{v'v} & \cdots & \underline{\mathbf{w}}_{N_{v'}}^{v'v} \\ | & | & & | & & | \end{pmatrix} \in \mathbb{R}^{(N_v+1)\times N_{v'}} \tag{3.2.10}$$

Therefore,

$$\underline{\mathbf{h}}^{v'} = \underline{\mathbf{W}}^{v'v\,\top} \cdot \underline{\mathbf{f}}^v\big(\underline{\mathbf{h}}^v\big) = \underline{\mathbf{W}}^{v'v\,\top} \cdot \underline{\mathbf{s}}^v \tag{3.2.11}$$

3.3 MLPs are universal function approximators

MLPs are universal function approximators. This means that, provided some assumptions are satisfied, they are capable of finding a function with which to map observations $\underline{\mathbf{x}} \in \mathbb{R}^N$ to a their corresponding label y_T.

$$y : \underline{\mathbf{x}} \mapsto y_T \tag{3.3.1}$$

Moving forward we will focus on *scalar* values for the label y_T but also explain how things can be extended to labels of larger dimensions.

The universal approximation theorem by Funahashi (1989)[1]

Let $y_T(\underline{\mathbf{x}})$ be a continuous, real valued function over a compact interval K and

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = \sum_{i=1}^{M} w_i^{21} f\Big(\sum_{j=1}^{N} w_{ij}^{10} \mathrm{x}_j - \theta_i\Big) \tag{3.3.2}$$

be a three-layered MLP with a non-constant, bounded, monotonously increasing and continuous function $f : \mathbb{R} \to \mathbb{R}$.

Then there exists a set of parameters $M, N \in \mathbb{N}$ and $w_i^{21}, w_{ij}^{10}, \theta_i \in \mathbb{R}$ such that for every $\varepsilon > 0$:

$$\max_{\underline{\mathbf{x}} \in K} \Big| y_T(\underline{\mathbf{x}}) - y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) \Big| \leq \varepsilon \tag{3.3.3}$$

3.4 Training a neural network

3.4.1 Recap of gradient-based learning

Training error E^T for the training set $\Big\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\Big\}$:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right) \tag{3.4.1}$$

The objective is to minimize the training error w.r.t. the model parameters $\underline{\mathbf{w}}$. That is

$$E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \quad \Rightarrow \quad \underline{\mathbf{w}}^* = \operatorname*{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} \tag{3.4.2}$$

Learning from gradient descent is an alternate approach for finding the minimum of a function when the closed-form solution is not available.

For an MLP:

$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\substack{\text{component of}\\ \text{gradient vector}}} \tag{3.4.3}$$

[1] Funahashi (1989) On the approximate realization of continuous mappings by neural networks. Neur Netw, 2:183–192
Hornik et al. (1989) Multilayer Feedforward Networks are Universal Approximators. Neur. Netw, 2:359–366.

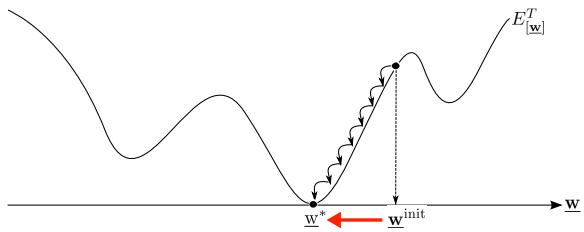


Figure 3.4.1: Minimizing the training error iteratively via gradient descent

The learning step (learning rate) η modulates the magnitude of our update. η can be treated as a constant but we will also see how the value of η can change over time, i.e. $\eta(t)$.

Q 3.4.1: *Do you remember why we <u>subtract</u> the gradient from $w_{ij}^{v'v}(t)$?*

- The gradient describes the slope. Subtracting the gradient aligns with minimizing the function, while adding the gradient would be suitable for maximizing it.

Recall from our previous discussion on calculating gradients for a simple connectionist neuron. Setting

- minimize quadratic error $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2$
- use connecitonist neuron as model class:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) := f\Big(\sum_{j=0}^{N} w_j x_j\Big) = f\left(h(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right) \tag{3.4.4}$$

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{the gradient}}} \tag{3.4.5}$$

$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}} \tag{3.4.6}$$

then specifically for quadratic error and the connectionist neuron:

$$= \frac{1}{p}\sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)}\right) \cdot f'(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})) \cdot x_j \tag{3.4.7}$$

3.4.2 Computing gradients for an MLP

Eq. (3.4.3) shows us how gradients are used for iteratively finding the minimum of the training error. The layered structure of an MLP implies that computing the gradient requires a more involved application of the chain rule. The backpropagation algorithm exploits the chain rule to efficiently compute the gradients for an MLP.

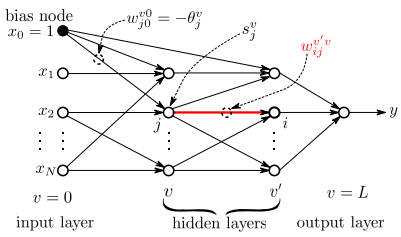


Figure 3.4.2: Example MLP architecture

3.5 The Backpropagation algorithm

The Backpropagation algorithm is a method for computing gradients by using the chain rule efficiently. Eq. (**??**) shows us how the gradient breaks down into (1) an error term and (2) a term dependent on the type of model chosen. Specifically:

$$\frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} \tag{3.5.1}$$

The Backpropagation algorithm handles the computation of the term in Eq. (3.5.1) for neural networks such as the one depicted in Fig. 3.4.2.

The Backpropagation algorithm is not tied to any specific neural architecture. We will therefore attempt to formulate how it works in the general sense.

Let $w_{ij}^{v'v}$ be the weight connecting neuron j in layer v to neuron i in layer v', In order to compute $\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}}$ we need to identify the following:

- $P(v, j)$: the set of immediate parent nodes feeding input into neuron (v, j),
- $C(v', i)$: the set of immediate children nodes which neuron (v', i) provides with input

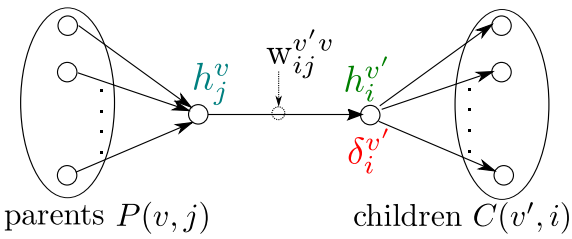


Figure 3.5.1: Nodes (v, j) and (v', i) are directly connected via the weight $w_{ij}^{v'v}$. The bias node for neuron (v, j) is included in the set of its parent nodes.

$h_i^{v'}$ measures the total input arriving at neuron i in layer v'. $h_i^{v'}$ is the weighted sum of this neuron's parent activations:

$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}(h_k^{\mu}), \tag{3.5.2}$$

where μ and k in Eq. (3.5.2) serve as indices for describing the position of each parent node in $P(v', i)$ in the network.

Q 3.5.1: *How does $w_{ij}^{v'v}$ contribute to the error?*

- The contribution is measured by applying the chain rule:

$$\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_i^{v'}}}_{\substack{:=\,\delta_i^{v'} \\ \text{“local error”} \\ \text{at neuron} \\ (v',i)}} \cdot \underbrace{\frac{\partial h_i^{v'}}{\partial w_{ij}^{v'v}}}_{\substack{=\,f_j^v(h_j^v) \\ \text{activity} \\ \text{of neuron} \\ (v,j)}} \tag{3.5.3}$$

Eq. (3.5.3) breaks the contribution of $w_{ij}^{v'v}$ to the network's output into two parts:

1. The contribution of $h_i^{v'}$ to the network's response $y(\underline{\mathbf{x}};\underline{\mathbf{w}})$. This is referred to as the "local error" at neuron (v', i). The efficiency of the Backpropagation algorithm is based on how it computes the "local error" denoted by $\delta_i^{v'}$ for neuron (v', i) and all other neurons in the network.
2. The contribution of $h_i^{v'}$ due to one of its inputs, namely neuron (v, j) which is weighted by $w_{ij}^{v'v}$. We already have everything to compute this term and that is by taking the definition of $h_i^{v'}$ in Eq.(3.5.2) and taking the derivative w.r.t. $w_{ij}^{v'v}$. This should be straightforward since only a single term in the sum depends on $w_{ij}^{v'v}$:

$$\frac{\partial h_i^{v'}}{\partial w_{ij}^{v'v}} = \frac{\partial}{\partial w_{ij}^{v'v}} \left(\sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big) \right) = \underbrace{f_j^v(h_j^v)}_{\substack{\text{activity} \\ \text{of neuron} \\ (v,j)}} \tag{3.5.4}$$

The Backpropagation algorithm is composed of two stages:

1. **forward propagation** (forward pass) : Given some input $\underline{\mathbf{x}}$, compute the activities of all neurons and the response of the network.
2. **backward propagation** (backward pass) : Given the response of the network, recursively calculate the local errors starting with the output node and work your way backward through the network.

3.5.1 Forward and backward propagation

1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\rightarrow$ parents)

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L) \tag{3.5.5}$$

$$\delta_i^{v'} = \frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial h_i^{v'}} \qquad \text{“local error” at neuron } (v', i) \tag{3.5.6}$$

$$= \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{w_{ki}^{v''v'} \cdot [f_i^{v'}]'(h_i^{v'})} \tag{3.5.7}$$

$$= \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} \cdot w_{ki}^{v''v'} [f_i^{v'}]'(h_i^{v'}) \tag{3.5.8}$$

$$= [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'} \tag{3.5.9}$$

weight update: using activities and local errors

$$w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial w_{ij}^{v'v}} = w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} \tag{3.5.10}$$

$$w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \delta_i^{v'} \cdot f_j^v(h_j^v) \tag{3.5.11}$$

To understand how the “local error” at neuron (v', i) goes from its definition in Eq. (3.5.6) to Eq. (3.5.7), one needs to look at the propagation from (v', i) forwards to all its immediate children nodes in $C(v', i)$. Let (v'', k) describe a neuron positioned deeper in the network and connected to (v', i), making it one of its children and, in turn, making (v', i) one of the parents of (v'', k). Analogous to how we compute in $h_i^{v'}$ in Eq.(3.5.2):

$$h_k^{v''} := \sum_{(\mu,l)\in P(v'',k)} w_{kl}^{v''\mu} \, f_l^\mu(h_l^\mu) \tag{3.5.12}$$

$$= \ldots + w_{ki}^{v''v'} \, f_i^{v'}(h_i^{v'}) + \ldots \tag{3.5.13}$$

Since only one term in this sum depends on $h_i^{v'}$:

$$\frac{\partial h_k^{v''}}{\partial h_i^{v'}} = \frac{\partial}{\partial h_i^{v'}} \left(w_{ki}^{v''v'} \, f_i^{v'}(h_i^{v'}) \right) \tag{3.5.14}$$

$$= w_{ki}^{v''v'} \frac{\partial}{\partial h_i^{v'}} f_i^{v'}(h_i^{v'}) \tag{3.5.15}$$

$$= w_{ki}^{v''v'} \, [f_i^{v'}]'(h_i^{v'}) \tag{3.5.16}$$

3.5.2 Example: MLP with fully connected layers

We will apply the Backpropagation algorithm to an MLP as depicted by Fig. 3.5.2. All neurons in one layer are connected with all nodes of the next layer. The layers are therefore *fully connected*. This implies that the weights between consecutive layers make up a *dense* weight matrix. This allows us to express the formulas using vector notation.

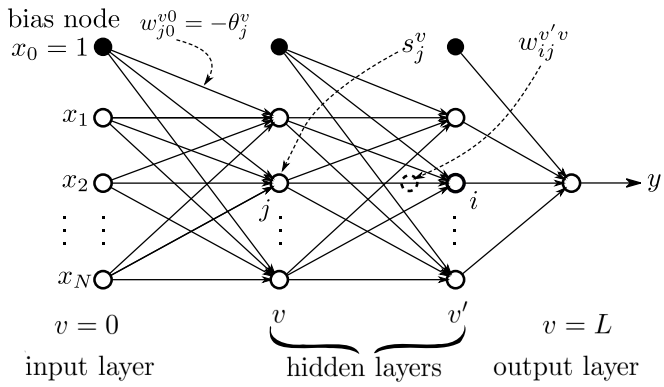


Figure 3.5.2: MLP with fully connected layers

Recall that the purpose of the Backpropagation algorithm is to compute the gradients. Therefore, the task is to obtain an expression for computing the gradients for all weights and biases in this network:

Q 3.5.2: *What is the procedure for the Backpropagation algorithm?*

- The procedure is:

1. forward pass: compute h_i^v for $v = 0, \ldots, L$,
2. backward pass: backpropagation of "local errors" (i.e. compute δ_i^v).

3.5.2.1 The forward pass

Compute h_i^v for $v = 0, \ldots, L$:

- $v = 0$ (input layer):

$$h_j^0 = x_j \,, \qquad j = 1, \ldots, N \tag{3.5.17}$$

$$h_0^0 = x_0 = 1 \qquad \text{(bias node)} \tag{3.5.18}$$

$$\underline{\mathbf{h}}^0 = \underline{\mathbf{x}} \qquad \text{(bias included)} \tag{3.5.19}$$

$$f_j^0(h) := h \qquad \text{(identity function)} \tag{3.5.20}$$

$$\underline{\mathbf{f}}^0(\underline{\mathbf{h}}) = \underline{\mathbf{x}} \qquad \text{(identity, bias included)} \tag{3.5.21}$$

- $v = 1$ (1st hidden layer with N_1 nodes):

$$h_j^1 = \sum_{k=0}^{N} w_{jk}^{10}\, f_k^0(h_k^0) = \underline{\mathbf{w}}_j^{10\top} \underline{\mathbf{h}}^0 \qquad \text{(bias included)} \tag{3.5.22}$$

$$\underline{\mathbf{h}}^1 = \underline{\mathbf{W}}^{10\top} \underline{\mathbf{h}}^0 = \underline{\mathbf{W}}^{10\top} \underline{\mathbf{x}} \qquad \text{(biases included)} \tag{3.5.23}$$

$$f_j^1(h) := \text{e.g. } \tanh(h), \qquad j = 1, \ldots, N_1 \tag{3.5.24}$$

$$f_0^1(h) := h \qquad \text{(identity for bias)} \tag{3.5.25}$$

$$\tag{3.5.26}$$

- $v = 2$ (2nd hidden layer with N_2 nodes):

$$h_i^2 = \sum_{j=0}^{N_1} w_{ij}^{21}\, f_j^1\big(h_j^1\big) = \underline{\mathbf{w}}_i^{21\top} \underline{\mathbf{f}}^1(\underline{\mathbf{h}}^1) \qquad \text{(bias included)} \tag{3.5.27}$$

$$\underline{\mathbf{h}}^2 = \underline{\mathbf{W}}^{21\top} \underline{\mathbf{f}}^1(\underline{\mathbf{h}}^1) \qquad \text{(biases included)} \tag{3.5.28}$$

$$f_i^2(h) := \text{e.g. } \tanh(h) \qquad i = \mathbf{1}, \ldots, N_2 \tag{3.5.29}$$

$$f_0^2(h) := h \qquad \text{(identity for bias)} \tag{3.5.30}$$

$$\tag{3.5.31}$$

- $v = L$ (output layer):

$$h_1^L = \sum_{i=0}^{N_2} w_{1i}^{L2}\, f_i^2\big(h_i^2\big) = \underline{\mathbf{w}}_1^{L2\top} \underline{\mathbf{f}}^2(\underline{\mathbf{h}}^2) \qquad \text{(bias included)} \tag{3.5.32}$$

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L) \tag{3.5.33}$$

$$f_1^L(h) := \text{e.g. logistic function (sigmoid)} \tag{3.5.34}$$

$$\tag{3.5.35}$$

3.5.2.2 The backward pass

Compute δ_i^v:

- $v = L$ (output layer):

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L) \tag{3.5.36}$$

$$f_1^L(h) = \text{e.g. logistic function (sigmoid)} \tag{3.5.37}$$

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L) \cdot 1 \qquad (C(L,1) \text{ is an empty set}) \tag{3.5.38}$$

- $v = 2$ (2nd hidden layer with N_2 nodes):

$$f_i^2(h) := \text{e.g. } \tanh(h), \qquad i = 1, \ldots, N_2 \tag{3.5.39}$$

$$f_0^2(h) := h \qquad \text{(identity for bias)} \tag{3.5.40}$$

$$\delta_i^2 = [f_i^2]'(h_i^2)\, \delta_1^L\, w_{1i}^{L2} \tag{3.5.41}$$

$$\underline{\delta}^2 = [\underline{\mathbf{f}}^2]'(\underline{\widetilde{\mathbf{h}}}^2) \odot (\underline{\widetilde{\mathbf{w}}}_1^{L2} \cdot \delta_1^L)^{\dagger} \tag{3.5.42}$$

$$\delta_0^2 = 1 \cdot \delta_1^L\, w_{10}^{L2} \qquad \text{(local error for bias)} \tag{3.5.43}$$

$$\tag{3.5.44}$$

$^{\dagger}\underline{\widetilde{\mathbf{w}}}_1^{L2} \mathrel{\widehat{=}} \underline{\mathbf{w}}_1^{L2}$ without the bias node.

$\underline{\widetilde{\mathbf{h}}}^2 \mathrel{\widehat{=}} \underline{\mathbf{h}}^2$ without the bias node. "$\odot$" stands for elementwise multiplication, while "$\cdot$" stands for the dot product.

- $v = 1$ (1st hidden layer with N_1 nodes):

$$f_j^1(h) := \text{e.g. } \tanh(h), \qquad j = 1, \ldots, N_1 \tag{3.5.45}$$

$$f_0^1(h) := h \qquad \text{(identity for bias)} \tag{3.5.46}$$

$$\delta_j^1 = [f_j^1]'(h_j^1) \sum_{k=1}^{N_2} \delta_k^2 \, w_{kj}^{21} \qquad \text{(bias } k = 0 \text{ excl.)} \tag{3.5.47}$$

$$\underline{\delta}^1 = [\underline{\mathbf{f}}^1]'(\underline{\widetilde{\mathbf{h}}}^1) \odot (\underline{\widetilde{\mathbf{W}}}^{21} \underline{\delta}^2)^{\ddagger} \tag{3.5.48}$$

$$\delta_0^1 = 1 \cdot \sum_{k=1}^{N_2} \delta_k^2 \, w_{k0}^{21} \qquad \text{(local error for bias)} \tag{3.5.49}$$

$$\tag{3.5.50}$$

- $v = 0$ (input layer):

$$f_j^0(h) := h \qquad \text{(identity function)} \tag{3.5.51}$$

$$\underline{\mathbf{f}}^0(\underline{\mathbf{h}}) := \underline{\mathbf{x}} \qquad \text{(bias } x_0 \text{ included)} \tag{3.5.52}$$

$$\delta_j^0 \qquad \text{(Not applicable to observed input)} \tag{3.5.53}$$

Table 1: Overview of dimensions

layer	parameters	total input	"local" errors
$v = 0$	N/A	$\underline{\mathbf{h}}^0 := \underline{\mathbf{x}}$ $\underline{\mathbf{h}}^0 \in \mathbb{R}^{N+1}$	N/A
$v = 1$	$\underline{\mathbf{w}}_j^{10} \in \mathbb{R}^{N+1}$ $\underline{\mathbf{W}}^{10} \in \mathbb{R}^{(N+1)\times N_1}$	$\underline{\mathbf{h}}^1 \in \mathbb{R}^{N_1+1}$ $\underline{\widetilde{\mathbf{h}}}^1 \in \mathbb{R}^{N_1}$	$\underline{\delta}^1 \in \mathbb{R}^{N_1}$
$v = 2$	$\underline{\mathbf{w}}_i^{21} \in \mathbb{R}^{N_1+1}$ $\underline{\mathbf{W}}^{21} \in \mathbb{R}^{(N_1+1)\times N_2}$ $\underline{\widetilde{\mathbf{W}}}^{21} \in \mathbb{R}^{(N_1)\times N_2}$	$\underline{\mathbf{h}}^2 \in \mathbb{R}^{N_2+1}$ $\underline{\widetilde{\mathbf{h}}}^2 \in \mathbb{R}^{N_2}$	$\underline{\delta}^2 \in \mathbb{R}^{N_2}$
$v = L$	$\underline{\mathbf{w}}_1^{L2} \in \mathbb{R}^{N_2+1}$	$h_1^L \in \mathbb{R}$	$\delta_1^L \in \mathbb{R}$

$^{\ddagger}\underline{\widetilde{\mathbf{W}}}^{21} \mathrel{\widehat{=}} \underline{\mathbf{W}}^{21}$ without the bias in each column vector $\underline{\mathbf{w}}_j^{21}$. ${w_{kj}}^{21}$ refers to row j, col. k of $\underline{\widetilde{\mathbf{W}}}^{21}$. $\underline{\widetilde{\mathbf{h}}}^1 \mathrel{\widehat{=}} \underline{\mathbf{h}}^1$ without the bias node.

3.5.3 Summary of the Backpropagation for gradient descent

initialization of weights and thresholds

while *stopping criterion not met* **do**

  $\text{gradient}_{ij}^{v'v} := 0\,, \quad \forall w_{ij}^{v'v}$

  for $\alpha \in \{1, \ldots, p\}$ **do**

    $h_i^0 := x_i^{(\alpha)}\,, \quad \forall i$ // forward propagation

    for $v' \in \{1, \ldots, L\}$ **do**

      $h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \underbrace{f_k^\mu\left(h_k^\mu\right)}_{x_k^{(\alpha)} \text{ if } v'=1}\,, \quad \forall i$

    end

    $\delta_1^L := [f_1^L]'(h_1^L)$ // backward propagation

    for $v' \in \{L-1, \ldots, 1\}$ **do**

      $\delta_i^{v'} := [f_i^{v'}]'(h_i^{v'}) \sum_{(\mu,k)\in C(v',i)} \delta_k^\mu\, w_{ki}^{\mu v'}\,, \quad \forall i$

    end

    $\text{gradient}_{ij}^{v'v} := \text{gradient}_{ij}^{v'v} + \frac{\partial e^{(\alpha)}}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}\, \delta_i^{v'}\, f_j^v\left(h_j^v\right), \quad \forall w_{ij}^{v'v}$ // sum

  end

  // gradient descent step:

  $\text{gradient}_{ij}^{v'v} := \frac{1}{p}\text{gradient}_{ij}^{v'v}\; w_{ij}^{v'v} := w_{ij}^{v'v} - \eta\, \text{gradient}_{ij}^{v'v}\,, \quad \forall w_{ij}^{v'v}$

end

3.5.3.1 Complexity

The computational and memory complexity of the Backpropagation algorithm is $\mathcal{O}(n)$, where n is the total number of the weights in the network. It is therefore, *linear* in the number of weights.

An alternativeand very simple method for computing the gradients would be to use *finite differences*.

Let $\underline{\widetilde{\mathbf{w}}}$ denote all parameters of the network except for a single weight $w_{ij}^{v'v}$

The idea is to add (subtract) a small offset ε to (from) $w_{ij}^{v'v}$ in the network, while keeping all other weights $\underline{\widetilde{\mathbf{w}}}$ fixed and measure the change in the error function. Specifically,

$$e\big(y_T, y\big(\underline{\mathbf{x}}; \underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big)\big) =: e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big) \text{ for brevity} \tag{3.5.54}$$

For $\varepsilon \ll 1$ follows:

$$\frac{\partial e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)}{\partial w_{ij}^{v'v}} = \frac{e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} + \varepsilon\big) + e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} - \varepsilon\big)}{2\,\varepsilon} + \mathcal{O}(\varepsilon^2) \tag{3.5.55}$$

The complexity of the forward propagation remains $\mathcal{O}(n)$. Performed it twice in Eq. (3.5.55) is manageable as $\mathcal{O}(2n) \rightsquigarrow \mathcal{O}(n)$. However, this is the complexity for a single weight. Performing this for all n weights in the network increases the complexity of finite differences to $\mathcal{O}(n^2)$.

However, numerical differentiation such as finite differences remains a useful tool for debugging your Backpropagation algorithm and performing gradient checks.

3.6 Symmetries

The layered structure of MLPs leads to *symmetries* in the parameter space.

For every $\underline{\mathbf{w}}$ there exists at least one other $\underline{\mathbf{w}}'$ which shares the exact same network response:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = y(\underline{\mathbf{x}}; \underline{\mathbf{w}}') \tag{3.6.1}$$

3.6.1 Permutation of indices within consecutive layers

We can ensure the same model response and training error E^T if:

$$w'^{\,v\,v-1}_{ij} = w^{v\,v-1}_{\Pi(i)j} \tag{3.6.2}$$

for some permutation π <u>and</u>

$$w'^{\,v+1\,v}_{ki} = w^{v+1\,v}_{k\pi(i)} \tag{3.6.3}$$

then neuron (v, i) behaves identically to neuron $(v, \pi(i))$

$N_v!$ is the total number of possible permutations per hidden layer v.

3.6.2 Sign-reversal across layers

We can ensure the same model response and training error E^T if:

$$w'^{\,v\,v-1}_{ij} = -\,w^{v\,v-1}_{ij}\,, \tag{3.6.4}$$

$$w'^{\,v+1\,v}_{ki} = -\,w^{v+1\,v}_{ki} \text{ and} \tag{3.6.5}$$

$$f^v(h) = f^{v+1}(h) := \tanh(h), \tag{3.6.6}$$

since $\tanh$ is an odd function: $\tanh(-h) = -\tanh(h)$.

2^{N_v} is the total number of sign-reversal combinations per hidden layer v.

Both types of symmetries give us a total of $\prod_{v=1}^{L} N_v!\,2^{N_v}$ eqivalent solutions (same response, same E^T). This is true for global and local minima. It implies that solutions of MLPs with 2 or more hidden layers are not unique.